



Unit 3 — Conditional Statements

Learning Module | Beginner-Friendly

LEARNING OBJECTIVES

After completing this module, you will be able to:

- Understand what conditional statements are and why they are used in programming
- Write programs using if, if-else, and if-elif-else statements
- Use comparison and logical operators to build conditions
- Apply nested conditional statements in real-world programs
- Build practical programs using all the conditional concepts

TABLE OF CONTENTS

1. Introduction to Conditional Statements	3
2. The if Statement	5
3. The if-else Statement	8
4. The if-elif-else Statement	11
5. Nested if Statements	14
6. Comparison Operators in Conditions	17
7. Logical Operators (and, or, not)	20
8. Practical Examples of Conditional Statements	24

1. Introduction to Conditional Statements

1.1 What is a Conditional Statement?

In everyday life, we make decisions constantly. For example, if it is raining, we carry an umbrella. If a bank balance is sufficient, a transaction is approved. These are examples of conditions — situations where an action depends on whether something is true or false.

In programming, a conditional statement is an instruction that tells the computer to check a condition and then execute a particular block of code depending on whether that condition is True or False.

Key Concept

A conditional statement allows your program to make decisions. Different code runs depending on whether a condition evaluates to True or False.

1.2 Why Do We Need Conditional Statements?

Without conditional statements, a program would follow the same path every single time it runs. It would have no ability to respond differently based on different inputs or situations. Consider a simple grade-checking program:

- Without conditions: The program prints the same message regardless of the marks entered.
- With conditions: The program prints different messages — 'Pass' or 'Fail' — based on the marks.

Conditional statements are the foundation of decision-making in programs. They are used in almost every real-world application — from login systems and banking software to games and search engines.

1.3 How Python Evaluates a Condition

Python evaluates a condition and produces one of two possible results:

Result	Meaning in Python
True	The condition is satisfied — execute the associated block
False	The condition is not satisfied — skip or take an alternate path

Let us see a very simple example of how Python decides True or False:

Example 1.1 — Evaluating a Condition

```
# Checking a simple condition in Python
temperature = 38

print(temperature > 35) # This condition is True
print(temperature > 40) # This condition is False
```

Output:

```
True
False
```

1.4 Python Indentation — A Critical Rule

Unlike many other programming languages that use curly braces { } to group code, Python uses indentation (spaces at the beginning of a line) to define blocks of code. Inside a conditional statement, all code that belongs to that block must be indented consistently.

Important Rule

Python uses 4 spaces (or 1 Tab) for indentation. Every line inside a conditional block must have the same indentation. If indentation is incorrect, Python will throw an `IndentationError`.

Example 1.2 — Understanding Indentation

```
# Correct indentation
marks = 75
if marks >= 50:
    print('You have passed.') # 4 spaces before print

# Incorrect indentation — this will cause an error!
if marks >= 50:
print('You have passed.') # No indentation — ERROR
```

2.1 Syntax of the if Statement

The if statement is the simplest form of a conditional statement in Python. It checks a condition, and if the condition is True, it executes the block of code inside it. If the condition is False, it simply skips that block.

Syntax — if Statement

```
if condition:
```

```
# code to execute if condition is True
```

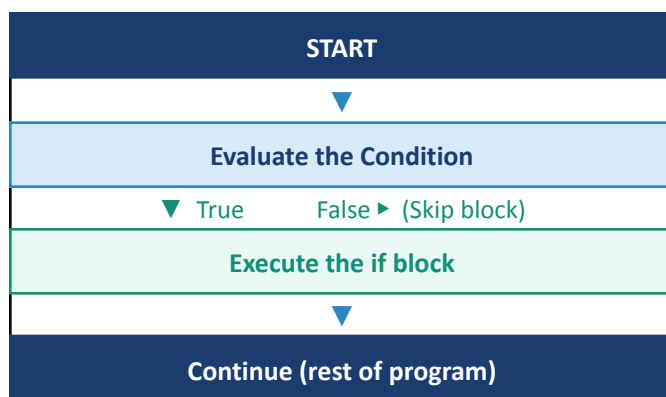
Remember

Notice the colon (:) at the end of the if line. This colon is mandatory. It tells Python that a block of code is about to follow. Missing the colon will result in a `SyntaxError`.

2.2 How the if Statement Works — Step by Step

When Python encounters an if statement, it follows these steps:

Flowchart: if Statement



2.3 Examples of the if Statement

Example 1: Checking a Number

Write a program to check if a number entered by the user is positive.

Example 2.1 — Checking for a Positive Number

```
# Program to check if a number is positive
number = int(input('Enter a number: '))

if number > 0:
    print('The number is positive.')

print('Program has ended.')
```

Output:

```
Enter a number: 7
The number is positive.
Program has ended.
```

Notice that when the user enters 7, the condition `number > 0` is `True`, so the message is printed. If the user enters -3, the condition is `False` and the message is skipped — but 'Program has ended.' is always printed because it is outside the `if` block.

Example 2: Verifying Age for Eligibility

Write a program to check if a person is eligible to vote (age must be 18 or above).

Example 2.2 — Voter Eligibility Check

```
# Voter eligibility check
age = int(input('Enter your age: '))

if age >= 18:
    print('You are eligible to vote.')
    print('Please visit your nearest polling booth.')

print('Thank you for using this system.')
```

Output:

```
Enter your age: 21
You are eligible to vote.
Please visit your nearest polling booth.
Thank you for using this system.
```

Example 3: Checking Account Balance

Write a program that checks if a bank account has enough balance to proceed with a transaction.

Example 2.3 — ATM Balance Check

```
# ATM balance check
balance = float(input('Enter your account balance: '))
withdrawal = float(input('Enter amount to withdraw: '))

if withdrawal <= balance:
    print('Transaction approved.')
    print('Please collect your cash.')
```

Output:

```
Enter your account balance: 5000
Enter amount to withdraw: 2000
Transaction approved.
```

Please collect your cash.



3. The if-else Statement

3.1 What is the if-else Statement?

The if statement only acts when a condition is True. But what if we also want the program to do something when the condition is False? That is exactly what the if-else statement does.

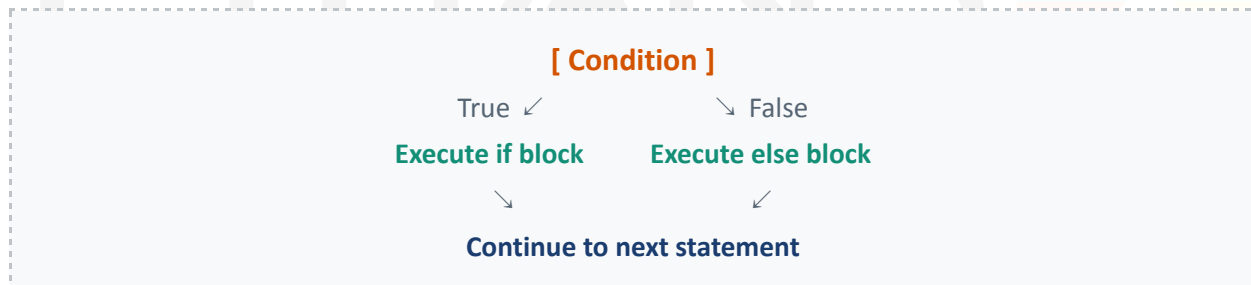
The if-else statement provides two paths: one for when the condition is True, and another for when the condition is False. Exactly one of the two blocks will execute — never both.

Syntax — if-else Statement

```
if condition:
    # code to execute if condition is True
else:
    # code to execute if condition is False
```

3.2 Flowchart — if-else

Flowchart: if-else Statement



3.3 Examples of the if-else Statement

Example 1: Pass or Fail

Write a program to check whether a student has passed or failed based on their marks (passing marks: 40).

Example 3.1 — Student Result Checker

```
# Student result checker
marks = int(input('Enter your marks: '))

if marks >= 40:
```

```
print('Result: PASS')
print('Congratulations! You have cleared the examination.')
else:
    print('Result: FAIL')
    print('Please prepare better and try again.')
```

Output:

```
Enter your marks: 35
Result: FAIL
Please prepare better and try again.
```

Example 2: Even or Odd Number

Write a program to determine whether a number entered by the user is even or odd.

Example 3.2 — Even or Odd Checker

```
# Even or Odd checker
number = int(input('Enter a number: '))

if number % 2 == 0:
    print(number, 'is an Even number.')
else:
    print(number, 'is an Odd number.')
```

Output:

```
Enter a number: 7
7 is an Odd number.
```

Concept Tip

The % symbol is the modulus operator. It returns the remainder when one number is divided by another. If a number divided by 2 gives a remainder of 0, it is even. Otherwise, it is odd.

Example 3: ATM Withdrawal with Insufficient Funds

Extending Example 2.3 — now we also show a message when the balance is insufficient.

Example 3.3 — ATM Withdrawal with Feedback

```
# ATM withdrawal with proper feedback
balance = float(input('Enter your account balance: '))
```



```
withdrawal = float(input('Enter amount to withdraw: '))

if withdrawal <= balance:
    balance = balance - withdrawal
    print('Transaction successful!')
    print('Remaining balance:', balance)
else:
    print('Insufficient balance.')
    print('Transaction declined. Please check your balance.')
```

Output:

```
Enter your account balance: 1000
Enter amount to withdraw: 1500
Insufficient balance.
Transaction declined. Please check your balance.
```

Example 4: Login System

Write a program that checks a username and password and either grants or denies access.

 Example 3.4 — Simple Login System

```
# Simple login system
correct_username = 'admin'
correct_password = 'python123'

username = input('Enter username: ')
password = input('Enter password: ')

if username == correct_username and password == correct_password:
    print('Login successful! Welcome,', username)
else:
    print('Login failed. Incorrect username or password.')
```

Output:

```
Enter username: admin
Enter password: python123
Login successful! Welcome, admin
```

4. The if-elif-else Statement

4.1 Limitations of if-else

The if-else statement handles two possibilities: True or False. However, real-world programs often require handling more than two situations. For example, a grade calculator might need to assign grades A, B, C, D, or F based on a range of marks. Using only if-else for this would become complicated.

Python solves this with the elif (short for else-if) keyword, which allows multiple conditions to be checked in sequence.

4.2 Syntax of if-elif-else

Syntax — if-elif-else Statement

```
if condition_1:
    # runs if condition_1 is True
elif condition_2:
    # runs if condition_1 is False AND condition_2 is True
elif condition_3:
    # runs if condition_1 and condition_2 are False AND condition_3 is True
else:
    # runs if ALL conditions above are False
```

How Python Processes elif

Python evaluates conditions top to bottom. As soon as one condition is True, it executes that block and skips all remaining elif and else blocks. Only one block is ever executed.

4.3 Examples of if-elif-else

Example 1: Grade Calculator

Write a program to assign a letter grade based on a student's marks.

Marks Range	Grade	Description
90 – 100	A+	Outstanding
80 – 89	A	Excellent
70 – 79	B	Very Good

60 – 69	C	Good
50 – 59	D	Satisfactory
Below 50	F	Fail

Example 4.1 — Grade Calculator

```
# Grade calculator
marks = int(input('Enter your marks (0-100): '))

if marks >= 90:
    grade = 'A+'
    remark = 'Outstanding'
elif marks >= 80:
    grade = 'A'
    remark = 'Excellent'
elif marks >= 70:
    grade = 'B'
    remark = 'Very Good'
elif marks >= 60:
    grade = 'C'
    remark = 'Good'
elif marks >= 50:
    grade = 'D'
    remark = 'Satisfactory'
else:
    grade = 'F'
    remark = 'Fail'

print('Grade:', grade)
print('Remark:', remark)
```

Output:

```
Enter your marks (0-100): 76
Grade: B
Remark: Very Good
```

Example 2: Day Classifier

Write a program that takes a day number (1–7) and prints the corresponding day name.

Example 4.2 — Day Classifier

```
# Day classifier
day_number = int(input('Enter a day number (1-7): '))

if day_number == 1:
    print('Monday')
elif day_number == 2:
    print('Tuesday')
elif day_number == 3:
    print('Wednesday')
elif day_number == 4:
    print('Thursday')
elif day_number == 5:
    print('Friday')
elif day_number == 6:
    print('Saturday')
elif day_number == 7:
    print('Sunday')
else:
    print('Invalid input. Please enter a number between 1 and 7.')
```

Output:

```
Enter a day number (1-7): 5
Friday
```

Example 3: Employee Tax Calculator

Write a program to calculate the tax rate based on an employee's annual income.

Example 4.3 — Tax Bracket Calculator

```
# Tax bracket calculator
income = float(input('Enter annual income: '))

if income <= 250000:
    tax_rate = 0
    tax_slab = 'No Tax'
elif income <= 500000:
```

```
tax_rate = 5
tax_slab = '5% Tax Slab'
elif income <= 1000000:
    tax_rate = 20
    tax_slab = '20% Tax Slab'
else:
    tax_rate = 30
    tax_slab = '30% Tax Slab'

tax_amount = (income * tax_rate) / 100
print('Tax Slab:', tax_slab)
print('Tax Amount:', tax_amount)
```

Output:

```
Enter annual income: 750000
Tax Slab: 20% Tax Slab
Tax Amount: 150000.0
```

ETIHANS
CLOUD & AI ACADEMY



5. Nested if Statements

5.1 What is a Nested if Statement?

A nested if statement is an if statement written inside another if statement. This allows the program to check a secondary condition only when a primary condition is already True (or False).

Think of it like a security system: first, you check if the main door is unlocked (outer condition). Only if it is unlocked do you then check if the correct PIN is entered (inner condition).

Syntax — Nested if Statement

```
if outer_condition:
    # code
    if inner_condition:
        # code that runs only when both conditions are True
    else:
        # code for outer True but inner False
else:
    # code for outer False
```

Caution

Nested conditions can become difficult to read when there are many levels of nesting. As a general guideline, avoid going beyond 2–3 levels of nesting. Deeply nested code is hard to debug and maintain.

5.2 Examples of Nested if Statements

Example 1: ATM PIN and Balance Check

A complete ATM scenario: first verify the PIN, then check if the balance is sufficient.

Example 5.1 — ATM PIN and Balance Check

```
# ATM simulation with nested if
correct_pin = 1234
balance = 10000

entered_pin = int(input('Enter your 4-digit PIN: '))

if entered_pin == correct_pin:
```

```
print('PIN verified successfully.')
amount = float(input('Enter withdrawal amount: '))

if amount <= balance:
    balance = balance - amount
    print('Transaction successful!')
    print('Remaining balance: ', balance)
else:
    print('Insufficient balance. Transaction declined.')
else:
    print('Incorrect PIN. Card blocked for security.')
```

Output:

```
Enter your 4-digit PIN: 1234
PIN verified successfully.
Enter withdrawal amount: 3000
Transaction successful!
Remaining balance: 7000.0
```

Example 2: University Admission Criteria

A student must first meet the minimum marks requirement. If they do, check whether they qualify for a scholarship.

Example 5.2 — Admission with Scholarship Check

```
# University admission with scholarship check
marks = int(input('Enter your marks (out of 100): '))

if marks >= 50:
    print('Admission granted!')

if marks >= 90:
    print('Scholarship: Full Scholarship awarded.')
elif marks >= 75:
    print('Scholarship: 50% fee waiver awarded.')
else:
    print('Scholarship: Not eligible for scholarship.')
else:
    print('Admission denied. Minimum 50 marks required.')
```

Output:

```
Enter your marks (out of 100): 82
Admission granted!
Scholarship: 50% fee waiver awarded.
```

Example 3: Employee Appraisal System

A company gives appraisals based on department and performance rating.

 Example 5.3 — Employee Appraisal System

```
# Employee appraisal system
department = input('Enter department (IT/Sales): ').strip()
rating = int(input('Enter performance rating (1-5): '))

if department == 'IT':
    if rating >= 4:
        print('Appraisal: 20% salary increment')
    elif rating == 3:
        print('Appraisal: 10% salary increment')
    else:
        print('Appraisal: No increment this year')
elif department == 'Sales':
    if rating >= 4:
        print('Appraisal: 15% increment + bonus')
    else:
        print('Appraisal: 8% increment')
else:
    print('Department not recognized.')
```

Output:

```
Enter department (IT/Sales): IT
Enter performance rating (1-5): 4
Appraisal: 20% salary increment
```


6. Comparison Operators in Conditions

6.1 What are Comparison Operators?

Comparison operators are used to compare two values. When Python evaluates a comparison, the result is always either True or False. These results are then used inside conditional statements to decide which code to run.

6.2 Complete Reference — Comparison Operators

Operator	Name	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	True
<code>!=</code>	Not equal to	<code>5 != 3</code>	True
<code>></code>	Greater than	<code>7 > 3</code>	True
<code><</code>	Less than	<code>3 < 7</code>	True
<code>>=</code>	Greater than or equal to	<code>5 >= 5</code>	True
<code><=</code>	Less than or equal to	<code>3 <= 7</code>	True

Common Mistake

Students often confuse `=` (assignment operator) with `==` (comparison operator). The single `=` assigns a value to a variable. The double `==` checks if two values are equal. Using `=` inside an if condition will cause a `SyntaxError`.

6.3 Detailed Examples of Each Operator

Example 6.1 — All Comparison Operators

```
# Demonstrating all comparison operators
a = 10
b = 20

print('a == b:', a == b) # Equal to
print('a != b:', a != b) # Not equal to
print('a > b:', a > b)   # Greater than
print('a < b:', a < b)   # Less than
print('a >= b:', a >= b) # Greater than or equal to
print('a <= b:', a <= b) # Less than or equal to
```

Output:

```
a == b: False
a != b: True
a > b: False
a < b: True
a >= b: False
a <= b: True
```

6.4 Practical Examples Using Comparison Operators

Example 1: Speed Limit Checker

 Example 6.2 — Speed Limit Checker

```
# Speed limit checker
speed_limit = 80
vehicle_speed = int(input('Enter vehicle speed (km/h): '))

if vehicle_speed <= speed_limit:
    print('Speed is within the limit. Drive safely!')
elif vehicle_speed <= 100:
    print('Warning: You are slightly over the speed limit.')
    print('Fine: Rs. 500')
else:
    print('ALERT: Dangerous speed detected!')
    print('Heavy fine: Rs. 2000 + License suspension.')
```

Output:

```
Enter vehicle speed (km/h): 110
ALERT: Dangerous speed detected!
Heavy fine: Rs. 2000 + License suspension.
```

Example 2: Product Stock Alert

 Example 6.3 — Inventory Stock Alert

```
# Inventory stock alert
product_name = input('Enter product name: ')
stock_quantity = int(input('Enter stock quantity: '))

if stock_quantity == 0:
    print(product_name, '- OUT OF STOCK. Raise purchase order immediately.')
elif stock_quantity <= 10:
```

```
print(product_name, '- LOW STOCK. Reorder soon.')  
elif stock_quantity <= 50:  
    print(product_name, '- Stock is adequate.')  
else:  
    print(product_name, '- Stock is sufficient.')
```

Output:

```
Enter product name: Laptop  
Enter stock quantity: 7  
Laptop - LOW STOCK. Reorder soon.
```

ETIHANS
CLOUD & AI ACADEMY



7. Logical Operators (and, or, not)

7.1 What are Logical Operators?

Comparison operators check a single condition at a time. However, in real-world programs, we often need to combine two or more conditions. Logical operators allow us to do exactly that.

Python provides three logical operators:

Operator	Meaning	Returns True when...
and	Logical AND	Both conditions are True
or	Logical OR	At least one condition is True
not	Logical NOT	The condition is False (reverses the result)

7.2 The and Operator

The and operator returns True only when ALL conditions connected by and are True. If even one condition is False, the entire expression is False.

Condition A	Condition B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Example 1: Loan Eligibility

A person must be at least 21 years old AND have a monthly salary of at least 25,000 to be eligible for a loan.

Example 7.1 — Loan Eligibility with and

```
# Loan eligibility checker
age = int(input('Enter your age: '))
salary = float(input('Enter your monthly salary: '))

if age >= 21 and salary >= 25000:
    print('Loan application approved!')
else:
    print('Loan application rejected.')
```

```
print('Requirements: Age >= 21 and Salary >= 25000')
```

Output:

```
Enter your age: 25
Enter your monthly salary: 30000
Loan application approved!
```

Example 2: Valid Marks Range**Example 7.2 — Valid Marks Range using and**

```
# Validate marks input
marks = int(input('Enter marks: '))

if marks >= 0 and marks <= 100:
    print('Valid marks entered.')
else:
    print('Invalid marks. Marks must be between 0 and 100.')
```

Output:

```
Enter marks: 150
Invalid marks. Marks must be between 0 and 100.
```

7.3 The or Operator

The or operator returns True when at least one of the conditions is True. It only returns False when ALL conditions are False.

Condition A	Condition B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Example 3: Holiday Checker

A day is a holiday if it is a Saturday OR a Sunday.

Example 7.3 — Holiday Checker using or

```
# Holiday checker
day = input('Enter the day: ').strip().lower()

if day == 'saturday' or day == 'sunday':
```

```
print('It is a holiday. Office is closed.')
else:
    print('It is a working day. Office is open.')
```

Output:

```
Enter the day: Sunday
It is a holiday. Office is closed.
```

Example 4: Emergency Discount

A customer is eligible for an emergency discount if they are a senior citizen OR they have a disability card.

Example 7.4 — Discount using or

```
# Emergency discount eligibility
age = int(input('Enter your age: '))
has_disability_card = input('Do you have a disability card? (yes/no): ')

if age >= 60 or has_disability_card == 'yes':
    print('You are eligible for a 20% emergency discount.')
else:
    print('Standard pricing applies.')
```

Output:

```
Enter your age: 45
Do you have a disability card? (yes/no): yes
You are eligible for a 20% emergency discount.
```

7.4 The not Operator

The not operator reverses the result of a condition. If a condition is True, not makes it False. If a condition is False, not makes it True.

Condition	not Condition
True	False
False	True

Example 5: Access Control

Example 7.5 — Access Control using not

```
# Access control with not
```

```
is_banned = False

if not is_banned:
    print('Access granted. Welcome to the system!')
else:
    print('Access denied. Your account has been banned.')
```

Output:

Access granted. Welcome to the system!

7.5 Combining Multiple Logical Operators

You can combine and, or, and not in a single condition. Python follows operator precedence: not is evaluated first, then and, then or. It is good practice to use parentheses to make the order of evaluation clear.

 Example 7.6 — Combined Logical Operators

```
# Online exam eligibility
# Student must be: enrolled AND (paid fees OR has scholarship) AND NOT suspended

is_enrolled = True
has_paid_fees = False
has_scholarship = True
is_suspended = False

if is_enrolled and (has_paid_fees or has_scholarship) and not is_suspended:
    print('Eligible to appear in the exam.')
else:
    print('Not eligible. Please check your status.')
```

Output:

Eligible to appear in the exam.

8. Practical Examples of Conditional Statements

In this section, we combine everything learned so far — if, if-else, if-elif-else, nested if, comparison operators, and logical operators — to build complete, real-world programs.

8.1 Program: Simple Calculator

Write a program that takes two numbers and an operator from the user (+, -, *, /) and performs the corresponding arithmetic operation.

Program 8.1 — Simple Calculator

```
# Simple Calculator
num1 = float(input('Enter first number: '))
operator = input('Enter operator (+, -, *, /): ')
num2 = float(input('Enter second number: '))

if operator == '+':
    result = num1 + num2
    print('Result:', num1, '+', num2, '=', result)
elif operator == '-':
    result = num1 - num2
    print('Result:', num1, '-', num2, '=', result)
elif operator == '*':
    result = num1 * num2
    print('Result:', num1, '*', num2, '=', result)
elif operator == '/':
    if num2 != 0:
        result = num1 / num2
        print('Result:', num1, '/', num2, '=', result)
    else:
        print('Error: Cannot divide by zero!')
else:
    print('Invalid operator entered.')
```

Output:

```
Enter first number: 15
Enter operator (+, -, *, /): /
Enter second number: 3
Result: 15.0 / 3.0 = 5.0
```


8.2 Program: Student Report Card

Write a complete report card program. The student enters marks for 3 subjects. The program calculates the average, assigns a grade, and prints whether they are promoted.

Program 8.2 — Student Report Card

```
# Student Report Card Generator
print('==== Student Report Card ====\\n')

name = input('Enter student name: ')
marks1 = float(input('Enter marks in Subject 1: '))
marks2 = float(input('Enter marks in Subject 2: '))
marks3 = float(input('Enter marks in Subject 3: '))

total = marks1 + marks2 + marks3
average = total / 3

# Determine grade
if average >= 90:
    grade = 'A+'
elif average >= 80:
    grade = 'A'
elif average >= 70:
    grade = 'B'
elif average >= 60:
    grade = 'C'
elif average >= 50:
    grade = 'D'
else:
    grade = 'F'

# Determine promotion status
if marks1 >= 40 and marks2 >= 40 and marks3 >= 40:
    status = 'PROMOTED'
else:
    status = 'DETAINED (failed in one or more subjects)'
```

```
# Display report
print('\n----- Report Card -----')
print('Student Name  : ', name)
print('Subject 1    : ', marks1)
print('Subject 2    : ', marks2)
print('Subject 3    : ', marks3)
print('Total Marks  : ', total)
print('Average     : ', round(average, 2))
print('Grade       : ', grade)
print('Status      : ', status)
print('-----')
```

Output:

```
===== Student Report Card =====
```

```
Enter student name: Rahul Sharma
Enter marks in Subject 1: 78
Enter marks in Subject 2: 65
Enter marks in Subject 3: 82
```

```
----- Report Card -----
Student Name  : Rahul Sharma
Subject 1    : 78.0
Subject 2    : 65.0
Subject 3    : 82.0
Total Marks  : 225.0
Average     : 75.0
Grade       : B
Status      : PROMOTED
-----
```

8.3 Program: E-Commerce Discount Calculator

An online store offers discounts based on the purchase amount and customer type (Regular / Premium).

Customer Type	Purchase Amount	Discount
Premium	Above Rs. 5000	20%
Premium	Rs. 2000 – 5000	15%
Premium	Below Rs. 2000	10%
Regular	Above Rs. 5000	10%
Regular	Rs. 2000 – 5000	5%

Regular	Below Rs. 2000	0%
---------	----------------	----

Program 8.3 — E-Commerce Discount Calculator

```
# E-Commerce Discount Calculator
customer_type = input('Enter customer type (Regular/Premium: ').strip()
purchase_amount = float(input('Enter purchase amount: '))

if customer_type == 'Premium':
    if purchase_amount > 5000:
        discount_percent = 20
    elif purchase_amount >= 2000:
        discount_percent = 15
    else:
        discount_percent = 10
elif customer_type == 'Regular':
    if purchase_amount > 5000:
        discount_percent = 10
    elif purchase_amount >= 2000:
        discount_percent = 5
    else:
        discount_percent = 0
else:
    print('Invalid customer type entered.')
    discount_percent = 0

discount_amount = (purchase_amount * discount_percent) / 100
final_amount = purchase_amount - discount_amount

print("\n===== Invoice Summary =====")
print('Customer Type :', customer_type)
print('Purchase Amount : Rs.', purchase_amount)
print('Discount      :', discount_percent, '%')
print('Discount Amount : Rs.', discount_amount)
print('Amount Payable : Rs.', final_amount)
print('=====')
```

Output:

```
Enter customer type (Regular/Premium): Premium
Enter purchase amount: 6000
```

```
===== Invoice Summary =====
Customer Type : Premium
Purchase Amount : Rs. 6000.0
Discount      : 20 %
Discount Amount : Rs. 1200.0
Amount Payable : Rs. 4800.0
=====
```

8.4 Program: Number Guessing Game

Write a simple number guessing game where the user guesses a number between 1 and 10, and the program tells them if they are correct, too high, or too low.

Program 8.4 — Number Guessing Game

```
# Number Guessing Game
import random

secret_number = random.randint(1, 10)
guess = int(input('Guess a number between 1 and 10: '))

if guess < 1 or guess > 10:
    print('Invalid input. Please enter a number between 1 and 10.')
elif guess == secret_number:
    print('Correct! You guessed the right number:', secret_number)
elif guess < secret_number:
    print('Too low! The correct number was:', secret_number)
else:
    print('Too high! The correct number was:', secret_number)
```

Output:

```
Guess a number between 1 and 10: 7
Too high! The correct number was: 4
```

MODULE SUMMARY

The table below summarises all the conditional statement types covered in this module.

Concept	Use Case	Key Syntax
if statement	Execute code only when a condition is True	if condition:
if-else statement	Choose between two paths (True or False)	if ... else:
if-elif-else	Choose between multiple paths	if ... elif ... else:
Nested if	Check secondary conditions inside an outer block	if inside if
Comparison Operators	Compare values (==, !=, >, <, >=, <=)	a == b, a > b
Logical: and	Both conditions must be True	cond1 and cond2
Logical: or	At least one condition must be True	cond1 or cond2
Logical: not	Reverse the result of a condition	not condition

What Comes Next?

In the next module, you will learn about Loops in Python — for loops and while loops — which allow you to repeat blocks of code. Conditional statements and loops together are the two most powerful tools in programming and are used in virtually every program you will ever write.